

myshell> _

Enterprise-Knowledge Intelligence Platform

OSSP · Operating Systems & System Programming · Weeks 1–6

`malloc()`

`strtok()`

`fork()`

`execvp()`

`waitpid()`

`chdir()`

`pipe()`

`dup2()`

PROGRESS

Weeks 1–6 are complete and verified

6 / 6

course weeks done

Built, tested and documented

76

checks passing

3 test suites: 21 + 32 + 23

742

lines of C

5 modules and 5 headers

0

memory errors or leaks

ASan, LeakSanitizer, UBSan

What ShellForge 6.0 can do

W1

REPL loop and
banner

W2

Input of any length

W3

Command parser

W4

Run programs

W5

Built-in commands

W6

Pipes and IPC

Mapped to four course outcomes

CO-1

Weeks 1–3

Shell as the user-space interface

REPL loop, the build pipeline, and command-line parsing with `strtok()` into an `execvp()`-ready `argv[]`.

CO-2

Weeks 4–5

Processes and built-ins

`fork()`, `execvp()`, `waitpid()` and exit status. `cd`, `pwd` and `env` run inside the shell because they change its own state.

CO-3

Week 6

Inter-process communication

Anonymous pipes with `pipe()` and `dup2()`, two-process pipelines, parent and child messages, EOF.

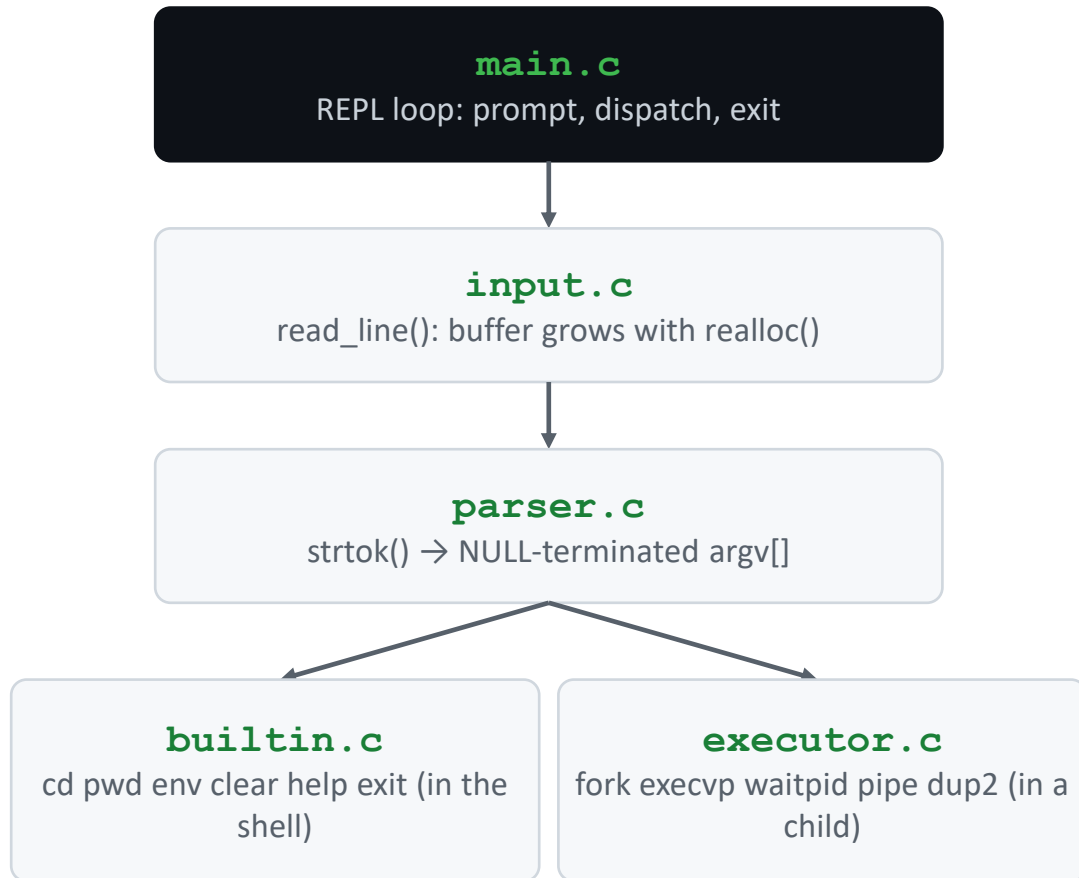
CO-4

Week 2 (foundation)

Dynamic memory

Heap buffers with `malloc()`, `realloc()` and `free()`; leaks and memory errors checked with AddressSanitizer.

Five small modules, one command loop



```
gcc:13 container: ShellForge source layout (C, 1,222 lines)

$ wc -l include/*.h src/*.c Makefile tests/*.sh
 29 include/builtin.h
 33 include/executor.h
  6 include/input.h
  7 include/parser.h
  8 include/shell.h
146 src/builtin.c
261 src/executor.c
 38 src/input.c
117 src/main.c
 97 src/parser.c
 27 Makefile
132 tests/test_week4.sh
179 tests/test_week5.sh
142 tests/test_week6.sh
1222 total
```

- Headers in include/, sources in src/, test scripts in tests/
- Weeks 1–5 each added one module; Week 6 extended the parser and executor
- 1,222 lines in total, including 453 lines of test scripts

Builds clean with gcc -Wall -Wextra

```
gcc:13 container: Build: make with gcc -Wall -Wextra (no warnings)

$ make clean && make
rm -f src/main.o src/input.o src/parser.o src/executor.o src/builtin.o
rm -rf bin
gcc -Wall -Wextra -g -Iinclude -c src/main.c -o src/main.o
gcc -Wall -Wextra -g -Iinclude -c src/input.c -o src/input.o
gcc -Wall -Wextra -g -Iinclude -c src/parser.c -o src/parser.o
gcc -Wall -Wextra -g -Iinclude -c src/executor.c -o src/executor.o
gcc -Wall -Wextra -g -Iinclude -c src/builtin.c -o src/builtin.o
gcc -Wall -Wextra -g -Iinclude -o bin/shellforge src/main.o src/input.o src/parser.o src/executor.o src/builtin.o
```

Makefile

make · make run · make test · make clean

Compiler flags

-Wall -Wextra -g for every build; the tests also build with AddressSanitizer and UBSan

Where it runs

Docker gcc:13 (Debian 12), because this Windows PC has no C compiler

Read a line, split it, hand it on

```

gcc:13 container: Weeks 1-3: REPL, dynamic input, strtok() parser (--debug-tokens)

$ ./bin/shellforge --debug-tokens
=====
Welcome to ShellForge Version 6.0
=====

myshell> ls -l /home

Parsed Tokens
argv[0] = ls
argv[1] = -l
argv[2] = /home
argv[3] = NULL
total 0
myshell> ls -l /home

Parsed Tokens
argv[0] = ls
argv[1] = -l
argv[2] = /home
argv[3] = NULL
total 0
myshell>
myshell> exit

Parsed Tokens
argv[0] = exit
argv[1] = NULL
Exiting ShellForge...

```

--debug-tokens prints argv[]; extra spaces collapse to the same three tokens.

W1

REPL loop

Banner, myshell> prompt, exit; Ctrl+D (end of input) exits cleanly. Modular Makefile build.

W2

Input of any length

read_line() starts small and grows with realloc(); a 3,000-character line arrives intact.

W3

Tokenizer

strtok() over spaces and tabs; argv[] grows 64 → 128 → 256 and ends with NULL, the shape execvp() needs.

Long input, no memory errors

```
gcc:13 container: Week 2: 3,000-character input under AddressSanitizer + UBSan

$ make clean >/dev/null && make CFLAGS="-Wall -Wextra -g -Iinclude -fsanitize=address,undefined" >/dev/null
$ cat longline.sh      # feeds a 3000-character command line: echo aaa...a | wc -c
{ printf 'echo '; head -c 3000 /dev/zero | tr '\0' a; printf ' | wc -c\nexit\n'; } | ./bin/shellforge
$ sh longline.sh

=====
Welcome to ShellForge Version 6.0
=====

myshell> 3001
myshell> Exiting ShellForge...
$ echo "exit status: $?"
exit status: 0
```

3,000 characters in

wc -c prints 3001: all 3,000 characters plus the newline

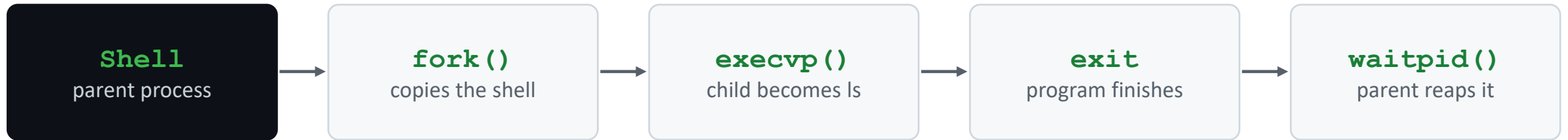
The buffer grows

malloc() to start, realloc() as the line gets longer, free() at exit

0 sanitizer reports

Built with -fsanitize=address,undefined; exit status 0

Every external command runs in its own process



```
gcc:13 container: Week 4: fork() / execvp() / waitpid()

$ ./bin/shellforge
=====
Welcome to ShellForge Version 6.0
=====

myshell> echo Hello from a child process
Hello from a child process
myshell> ls /usr
bin  games  include  lib  lib64  libexec  local  sbin  share  src
myshell> date
Fri Sep 18 08:27:47 UTC 2026
myshell> whoami
root
myshell> nosuchcommand
ShellForge: nosuchcommand: No such file or directory
myshell> exit
Exiting ShellForge...
```

- Exit status read with WIFEXITED and WEXITSTATUS; signals with WIFSIGNALED
- If execvp() fails, the child calls _exit(), so a second copy of the shell never keeps running
- stdout and stderr are flushed before fork(), so buffered text is not printed twice
- Unknown commands print an error and the shell keeps running

Built-ins change the shell itself

```

gcc:13 container: Week 5: built-in commands and environment variables

$ ./bin/shellforge
=====
Welcome to ShellForge Version 6.0
=====

myshell> help

ShellForge built-in commands
-----
cd [dir]  change the shell's working directory (defaults to $HOME)
pwd       print the working directory
env       show common environment variables
clear     clear the screen
help     show this list
exit     leave the shell

Anything else is executed as an external program via fork/execvp.

myshell> pwd
/src
myshell> cd /tmp
myshell> pwd
/tmp
myshell> env
HOME=/root
USER=(not set)
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
SHELL=(not set)
PWD=/tmp
myshell> cd
myshell> pwd
/root
myshell> cd /no/such/dir
ShellForge: cd: /no/such/dir: No such file or directory
myshell> cd a b
ShellForge: cd: too many arguments
myshell> exit
Exiting ShellForge...

```

Why cd can't be forked

The working directory belongs to each process. A child's `chdir()` would vanish when it exits, so `cd` has to run inside the shell. The tests prove it: `cd /tmp`, then `pwd` prints `/tmp`.

cd

pwd

env

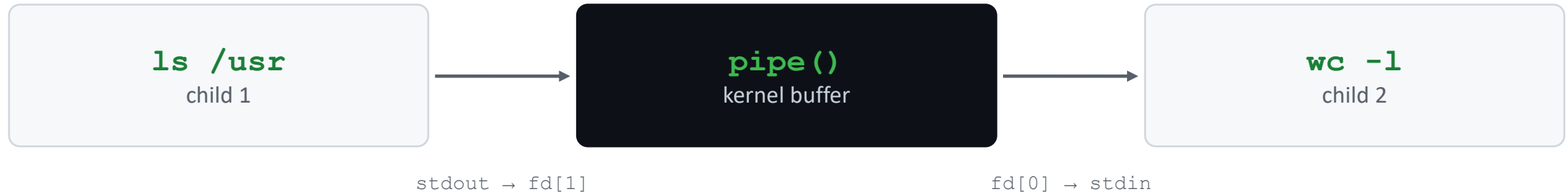
clear

help

exit

- PWD is updated on every `cd`
- Unset variables print (not set) instead of crashing
- Errors handled: missing directory, too many arguments
- Anything else still goes to `fork()` and `execvp()`

Two programs, one pipe



```

gcc:13 container: Week 6: pipes, cmd1 | cmd2 via pipe() and dup2()

$ ./bin/shellforge
=====
Welcome to ShellForge Version 6.0
=====

mysHELL> ls /usr | wc -l
10
mysHELL> echo hello world from shellforge | wc -w
4
mysHELL> cat /etc/os-release|head -2
PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"
NAME="Debian GNU/Linux"
mysHELL> ls /usr | grep lib
lib
lib64
libexec
mysHELL> exit
Exiting ShellForge...
  
```

- `dup2()` connects each child to its end of the pipe
- Works with or without spaces around the pipe symbol
- Every unused pipe end is closed, so the reader sees end-of-file and nothing hangs
- A leading or trailing `|` is a syntax error; pipelines have two stages, as this chapter specifies

Talking between processes

```
gcc:13 container: Week 6: IPC demo, parent <-> child over pipe(), EOF

$ ./bin/shellforge --demo-ipc

=====
ShellForge IPC Demonstration (CO-3: pipe() & IPC)
=====

[IPC Demo 1] Parent -> Child Unidirectional Communication
  [Parent PID 94] Sending to child: "Hello Child from Parent!"
  [Child PID 95] Received from parent: "Hello Child from Parent!"

[IPC Demo 2] Child -> Parent Unidirectional Communication
  [Child PID 96] Sending to parent: "Greetings Parent from Child!"
  [Parent PID 94] Received from child: "Greetings Parent from Child!"

[IPC Demo 3] EOF Detection Verification
  Closing write-end of pipe (pipe3[1])...
  Reader read() returned: 0 byte(s) (0 indicates EOF received)

=====
IPC Demonstration Completed Successfully
=====
```

1

Parent → child

The parent writes, the child reads; each closes the end it does not use.

2

Child → parent

The same idea with the pipe in the other direction.

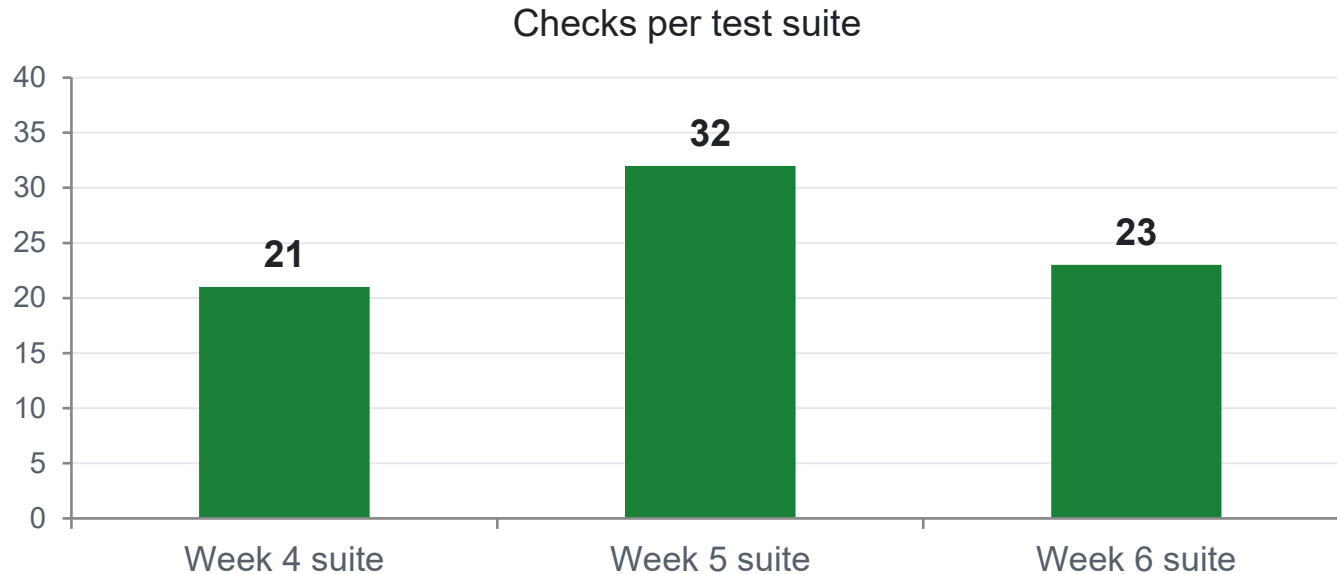
3

End-of-file

Once every write end is closed, read() returns 0.

Run it with `./bin/shellforge --demo-ipc`, or type `demo-ipc` at the prompt.

76 automated checks, all passing



- `make test` runs all three suites in the `gcc:13` container
- Each suite rebuilds the shell with AddressSanitizer and UBSan, and fails on any report
- Error paths are tested too: bad commands, bad `cd`, pipe syntax errors

gcc:13 container: make test: Week 6 suite (compile lines omitted)

```
$ make test
=====
Running ShellForge Week 6 Test Suite
=====
--- 1. Compilation Test ---
[PASS] Build under -Wall -Wextra
--- 2. Banner & Version Test ---
[PASS] Startup banner Version 6.0
[PASS] Clean exit message
--- 3. IPC Demonstrations ---
[PASS] Parent -> Child IPC
[PASS] Child -> Parent IPC
[PASS] EOF Detection (0 bytes)
--- 4. Interactive demo-ipc Command ---
[PASS] Interactive demo-ipc command
--- 5. Basic Two-Process Pipelines ---
[PASS] echo | grep pipeline
[PASS] seq | wc -l pipeline
--- 6. Pipeline Tokenization without Spaces ---
[PASS] Unspaced pipe cmd1|cmd2
--- 7. Multi-argument Pipeline ---
[PASS] ls -la bin | grep shellforge
--- 8. Pipeline Syntax Error Handling ---
[PASS] Leading pipe syntax error
[PASS] Trailing pipe syntax error
--- 9. Multi-stage Pipeline Boundary Check ---
[PASS] Multi-stage pipeline boundary rejection
--- 10. Pipeline Error Handling ---
[PASS] Invalid left command error
[PASS] Invalid right command error
--- 11. EOF Termination ---
[PASS] Empty input EOF exit code (exit code: 0)
--- 12. Dynamic Buffer & Token Growth in Pipeline ---
[PASS] Long argument in pipeline (>1024 chars)
[PASS] Large argument vector in pipeline
--- 13. Memory & Sanitizer Verification ---
[PASS] ASan IPC demo
[PASS] ASan pipeline execution
[PASS] ASan pipeline output count
[PASS] ASan clean exit
=====
Test Summary: 23 passed, 0 failed
=====
```

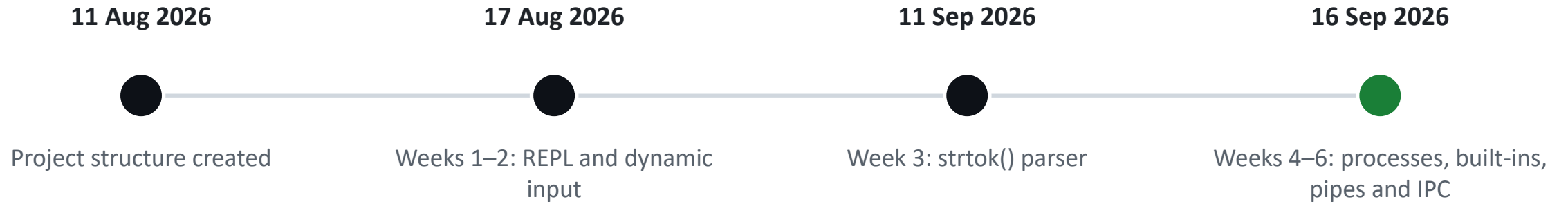
Where ShellForge differs from the handbook

Handbook listing	ShellForge	Why
Week 3: <code>#include "...../include/..."</code> has a stray quote, and the Makefile target has no prerequisites	Keeps <code>-include</code> and an object-file Makefile	The listing does not compile, and its target never rebuilds
Week 5 <code>env: printf("%s", getenv("USER"))</code>	Prints (not set) when a variable is unset	Passing NULL to printf is undefined behaviour
Week 5 <code>clear: system("clear")</code>	Writes the ANSI clear-screen code	No extra shell process, and clear is missing from many images
Week 5 <code>exit: calls exit() directly</code>	Returns to <code>main()</code> , which frees memory first	Keeps the leak checker clean

Also: pipes and IPC were first filed as Week 5. They were relabelled Week 6 to match the handbook; the code did not change.

HISTORY

Built in four commits



```
gcc:13 container: Git history of subjects/OSSP

$ git log --format="%h %ad %s" --date=short -- subjects/OSSP
0376f33 2026-09-16 feat(oss): Week 5 built-in commands, and correct the Week 6 chapter label
8325192 2026-09-11 feat(oss): Week 3 - command parser using strtok
687ab75 2026-08-17 OSSP Week2 Update
9937542 2026-08-11 chore: initialize multi-subject project architecture
```

NEXT

What's left in the syllabus

CO-3

Signals and job control

Signals, named pipes (FIFOs), process groups, sessions

CO-4

Memory

Virtual memory, mmap, page faults, memory debugging

CO-5

Files and I/O

File descriptors, open/read/write/close, directories, buffered I/O

CO-6

Threads and sync

POSIX threads, mutexes, condition variables, semaphores, deadlock

Week 7 starts with the next handbook chapter.

```
myshell> exit
```

Test logs: Weeks 4, 5 and 6

```

gcc:13 container: make test: Week 4 suite (compile lines omitted)

$ make test
=====
Running ShellForge Week 4 Test Suite
=====
--- 1. Compilation Test ---
[PASS] Build under -Wall -Wextra
--- 2. Banner & Version Test ---
[PASS] Startup banner
[PASS] Clean exit message
--- 3. Basic Command Execution ---
[PASS] Echo command execution
[PASS] Pwd command execution
--- 4. Multi-argument Execution ---
[PASS] Multiple arguments
[PASS] ls -la bin execution
--- 5. Non-existent Command Handling ---
[PASS] Command not found error
[PASS] Shell survives invalid command and exits
--- 6. Child Non-Zero Exit Handling ---
[PASS] Shell continues after child non-zero exit
--- 7. Whitespace Handling ---
[PASS] Blank lines ignored
--- 8. EOF Termination ---
[PASS] Empty input EOF exit code (exit code: 0)
--- 9. Long Input Test (>1024 characters) ---
[PASS] 1500-char input execution
--- 10. Large Argument Vector (200 tokens) ---
[PASS] 200-argument token 1
[PASS] 200-argument token 200
--- 11. Debug Tokens Mode ---
[PASS] Debug tokens argv[0]
[PASS] Debug tokens argv[1]
[PASS] Debug tokens argv[2]
[PASS] Debug tokens NULL termination
--- 12. Memory & Sanitizer Verification ---
[PASS] ASan clean execution
[PASS] ASan clean exit
=====
Test Summary: 21 passed, 0 failed
=====

```

```

gcc:13 container: make test: Week 5 suite (compile lines omitted)

$ make test
=====
Running ShellForge Week 5 Test Suite
(Built-in Commands & Environment Vars)
=====
--- 1. Compilation Test ---
[PASS] Build under -Wall -Wextra
--- 2. pwd Built-in ---
[PASS] pwd prints working directory
--- 3. cd Persists in the Parent Process ---
[PASS] cd /tmp then pwd shows /tmp
[PASS] cd did not silently stay in /src
[PASS] successive cd calls accumulate
--- 4. cd Defaults to $HOME ---
[PASS] bare cd returns to $HOME (/root)
--- 5. cd Error Handling ---
[PASS] cd to missing directory reports error
[PASS] shell survives a failed cd
[PASS] cd rejects too many arguments
--- 6. PWD Environment Variable Updated by cd ---
[PASS] env reports PWD=/tmp after cd
--- 7. env Built-in ---
[PASS] env prints HOME
[PASS] env prints PATH
[PASS] env prints SHELL
--- 8. env NULL-safety for Unset Variables ---
[PASS] unset USER renders as (not set)
--- 9. help Built-in ---
[PASS] help lists cd
[PASS] help lists pwd
[PASS] help lists env
[PASS] help lists clear
[PASS] help lists exit
--- 10. clear Built-in ---
[PASS] clear emits ANSI erase-display
--- 11. exit Built-in via Parser ---
[PASS] exit terminates cleanly
[PASS] exit with surrounding whitespace terminates
[PASS] nothing runs after exit
--- 12. External Commands Still Execute ---
[PASS] external echo still works
[PASS] unknown command falls through to execvp
[PASS] shell survives unknown command
--- 13. EOF Termination ---
[PASS] Empty input EOF exit code (exit code: 0)
--- 14. Memory & Sanitizer Verification ---
[PASS] ASan built-in run completed
[PASS] ASan clean exit
[PASS] No AddressSanitizer errors
[PASS] No LeakSanitizer leaks
[PASS] No UBSan runtime errors
=====
Test Summary: 32 passed, 0 failed
=====

```

```

gcc:13 container: make test: Week 6 suite (compile lines omitted)

$ make test
=====
Running ShellForge Week 6 Test Suite
=====
--- 1. Compilation Test ---
[PASS] Build under -Wall -Wextra
--- 2. Banner & Version Test ---
[PASS] Startup banner Version 6.0
[PASS] Clean exit message
--- 3. IPC Demonstrations ---
[PASS] Parent -> Child IPC
[PASS] Child -> Parent IPC
[PASS] EOF Detection (0 bytes)
--- 4. Interactive demo-ipc Command ---
[PASS] Interactive demo-ipc command
--- 5. Basic Two-Process Pipelines ---
[PASS] echo | grep pipeline
[PASS] seq | wc -l pipeline
--- 6. Pipeline Tokenization without Spaces ---
[PASS] Unspaced pipe cmd1|cmd2
--- 7. Multi-argument Pipeline ---
[PASS] ls -la bin | grep shellforge
--- 8. Pipeline Syntax Error Handling ---
[PASS] Leading pipe syntax error
[PASS] Trailing pipe syntax error
--- 9. Multi-stage Pipeline Boundary Check ---
[PASS] Multi-stage pipeline boundary rejection
--- 10. Pipeline Error Handling ---
[PASS] Invalid left command error
[PASS] Invalid right command error
--- 11. EOF Termination ---
[PASS] Empty input EOF exit code (exit code: 0)
--- 12. Dynamic Buffer & Token Growth in Pipeline ---
[PASS] Long argument in pipeline (>1024 chars)
[PASS] Large argument vector in pipeline
--- 13. Memory & Sanitizer Verification ---
[PASS] ASan IPC demo
[PASS] ASan pipeline execution
[PASS] ASan pipeline output count
[PASS] ASan clean exit
=====
Test Summary: 23 passed, 0 failed
=====

```